



Assignment Title:

Dynamic Robotics Simulation and Autonomous System Design

Programme title:

MSc Artificial Intelligence

Prepared by:

Amaan Mohammed Khalander

Module:

Robotics Software and Programming

Year:

Term 2 – 2026

Submission Date:

23 / September / 2026

Issued by:

Dr. Giulio Napolitano, Dr. Syed Arslan Abbas Rizvi

Learner declaration

Item	Response
Word count	3,286
Use of proof-reader/proof-reading service	NO
If yes, specify which service	N/A
I confirm that I submit this work as my own work and have cited all sources used	YES
I confirm that I have followed guidance on the acceptable use of AI tools for this assignment	YES
Where use of appropriately cited AI tools is permitted, I confirm that I have cited in accordance with the UCA Harvard Referencing Standard	YES

Student signature	Date
	23 / September / 2026

Source Code and Relevant Project Links:

Source Code:

- Task 1:
<https://github.com/Amaan9136>
- Task 2:
<https://github.com/Amaan9136>

Table of Contents

Abstract	6
Introduction.....	7
Task 1: Interactive PyBullet Environment with Object Semantics.....	8
3.1 Project Definition.....	8
3.2 Literature Review.....	8
3.3 Design and Implementation	9
3.4 Code Implementation.....	10
3.5 Execution and Testing.....	11
3.6 Evaluation and Reflection.....	12
Task 2: Goal-Oriented Autonomous Robot with Monitoring (ROS2).....	14
4.1 Project Definition.....	14
4.2 Design and Implementation	14
4.3 Execution and Testing.....	17
4.4 Evaluation and Reflection.....	21
Combined Evaluation and Reflection	23
Concluding Remarks.....	24
Acknowledgement	25
References.....	26

Table of Figures

Figure 1: Task 1 pipeline, from the floor-plan sketch and URDF generation through the semantic layer and camera rendering to the query helpers and outputs.	9
Figure 2: Six PyBullet camera viewpoints, robot repositioned between renders while all other objects remain fixed.	12
Figure 3: ROS2 node and topic architecture of the semantic_nav_monitor package, showing operator input, the four nodes and the Gazebo simulation.	15
Figure 4: Six-state finite state machine in mission_controller, with the REVERSE, ORIENT and COMMIT sub-phases shown beneath their parent states.	16
Figure 5: TheConstruct.ai Rosject XctZpczWOA: file explorer, mission_controller.py in the code editor, the live Simulation panel, and monitor_node's summary stream in the shell.	18
Figure 6: Gazebo client on TheConstruct.ai: robot near the charging dock among the eight pallet obstacles and checkpoint markers, with elapsed real time and iteration count visible. .	18
Figure 7: keyboard_hri_node in a Rosject shell: start and pause keypresses published and confirmed, alongside the live Simulation panel.	19
Figure 8: Live /mission_state echo showing the robot cycle through AVOID_OBSTACLE, REPLAN, IDLE, PLANNING, NAVIGATE and MISSION_COMPLETE.	19
Figure 9: monitor_node's mission summary, captured while the mission was in progress.	20
Figure 10: Local Gazebo Harmonic verification: LIDAR rays visualised around the robot near a pallet obstacle, with the forward camera feed shown alongside.	21

List of Tables

Table 1: Semantic object registry as printed by print_object_roles() (14 objects, excluding the robot).	10
Table 2: ROS2 node architecture.	15
Table 3: Finite state machine (six states).	16
Table 4: Mission summary logged by monitor_node on TheConstruct.ai, Rosject XctZpczWOA, while the mission was in progress.	20

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work, and that none of its parts has been copied from printed or electronic sources, translated from foreign sources, or reproduced from essays by other researchers or students. Wherever I have based my work on ideas or other people's texts, I clearly declare it through proper references in line with academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname:

AMAAN MOHAMMED KHALANDER

A handwritten signature in black ink, appearing to read 'Amaan MK', with a stylized, cursive script.

Date: 23 / September / 2026

Abstract

This report covers two practical exercises for the Robotics Software and Programming module. Task 1 is a PyBullet simulation of a small inspection and light-warehousing room. It uses five custom URDF object categories, a two-wheel robot, fourteen placed objects and six camera viewpoints. A dictionary called `semantic_db` stores each object's role and supports role lookup and nearest-target queries. Task 2 is a TurtleBot3 patrol robot in ROS2 with a six-state finite state machine. It runs in a custom Gazebo warehouse world with eight pallet obstacles and four checkpoints, one of which is the charging dock. The robot uses path planning, LIDAR and odometry-based obstacle avoidance and a keyboard interaction node. A monitoring node logs state transitions and mission metrics. Task 2 was run to `MISSION_COMPLETE` on TheConstruct.ai in Rosject XctZpczWOA. A local ROS2 Jazzy and Gazebo Harmonic workspace was used for extra checks.

Introduction

The module asks us to design, build and test robot software in simulation, with no physical hardware. The brief, Dynamic Robotics Simulation and Autonomous System Design, sets out two stages of increasing complexity: an interactive PyBullet environment with object-level semantics, and a ROS2-based autonomous system with sensing, navigation, human interaction and self-monitoring. The two stages use different tools, but the guidelines ask for one report, so both are covered here using the BSBI template.

In Task 1, every object in the simulated room gets a role (obstacle, target, landmark, shelf or charger) that is stored and can be looked up in code. Task 2 applies the idea to a moving robot that reads LIDAR and odometry, switches between behaviour states, and reports what it did and why.

Task 1: Interactive PyBullet Environment with Object Semantics

3.1 Project Definition

The objective of Task 1 was a PyBullet scene where code could look up the role of each object as well as its position. I sketched the room on paper before writing any code. It was a six by four metre space with a single entrance, a central aisle, storage shelving along the back wall, a charging dock near the entrance, and reference landmarks at each corner. I then turned the sketch into URDF categories and coordinates, so every object sits where I placed it.

The finished scene has fourteen objects in five URDF categories, which is above the brief's minimum of eight objects and four models. It also has a two-wheel robot that I moved between camera renders to show movement. All objects are URDF boxes and cylinders with no imported meshes, so the notebook is self-contained. I considered richer datasets such as SceneNet RGB-D (McCormac et al., 2017) but left them for future work.

3.2 Literature Review

Three areas of work informed Task 1. These were simulators, semantic representation and labelled scenes.

- **Simulators.** PyBullet exposes the Bullet physics engine through Python, loads URDF models and includes a CPU renderer (Coumans and Bai, 2021; PyBullet, 2023), so the notebook runs headless in Colab. Collins et al. (2021) review simulators across robotics. Task 1 needs only scene building and rendering, so PyBullet's light setup was enough.
- **Semantic representation.** Kostavelis and Gasteratos (2015) survey semantic mapping for mobile robots, noting its role in human-robot interaction, knowledge representation and planning. The registry I built is a very simple version of that idea. I assign the labels by hand, so code can ask what an object is as well as where it is.
- **Labelled scenes.** McCormac et al. (2017) generated SceneNet RGB-D, a large synthetic indoor dataset with ground-truth labels. Task 1 is a much smaller, hand-made scene that trades realism for full control over the labels. Duckett et al. (2018) note that field robots must interpret unstructured surroundings reliably.

Because I assign the labels, Task 1 shows how role queries work and does not test perception (Section 3.6).

3.3 Design and Implementation

The notebook (Task1_PyBullet_Object_Semantics.ipynb) has nine executable cells. The setup cell imports pybullet, pybullet_data, numpy and matplotlib, and installs pybullet if it is missing, so the notebook runs in Google Colab or locally. The generation cell writes six URDF files to disk. These are an obstacle (crate), a target zone, a landmark, a shelf, a charging dock and a two-wheel robot with a box chassis and two cylindrical wheels. The environment cell connects to PyBullet in DIRECT mode and loads a ground plane from the bundled pybullet_data assets. It then places the fourteen objects and the robot at the coordinates from my floor sketch. These are four crate obstacles flanking the aisle, three target zones (pickup, an off-aisle inspection point and drop-off), four corner landmarks, two back-wall shelves and one charging dock. Figure 1 shows the pipeline.

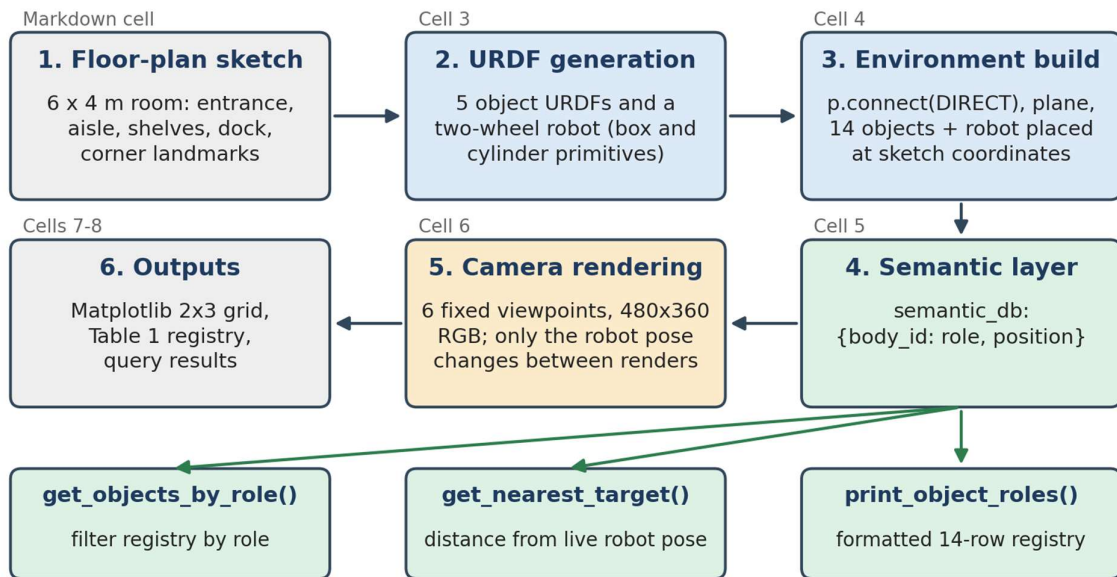


Figure 1: Task 1 pipeline, from the floor-plan sketch and URDF generation through the semantic layer and camera rendering to the query helpers and outputs.

The semantic layer, `semantic_db`, is a plain Python dictionary. Each PyBullet body id maps to a role string and an (x, y, z) position. Table 1 shows it as printed. Three helper functions use it. `get_objects_by_role` filters by role string. `get_nearest_target` measures the Euclidean distance from the robot's current base position to every target and returns the closest. `print_object_roles` prints the whole registry.

The rendering cell sets up six camera viewpoints with `computeViewMatrix` and `computeProjectionMatrixFOV` (PyBullet, 2023). The brief requires three, and I rendered six as an extension. Only the robot moves between renders and every other object stays fixed. `getCameraImage` then captures a 480 by 360 RGB array for each viewpoint.

I used Python 3, PyBullet, NumPy and Matplotlib. The notebook has no perception model. Roles are set when the scene is built, so it cannot work out a role from an image (see Section 3.6).

Table 1: Semantic object registry as printed by `print_object_roles()` (14 objects, excluding the robot).

body_id	role	position (x, y, z) m
1	obstacle	(1.50, 0.80, 0.25)
2	obstacle	(1.50, -0.80, 0.25)
3	obstacle	(3.50, 0.80, 0.25)
4	obstacle	(3.50, -0.80, 0.25)
5	target	(0.50, 0.00, 0.01)
6	target	(3.00, 1.60, 0.01)
7	target	(5.50, 0.00, 0.01)
8	landmark	(0.30, 1.80, 0.70)
9	landmark	(0.30, -1.80, 0.70)
10	landmark	(5.70, 1.80, 0.70)
11	landmark	(5.70, -1.80, 0.70)
12	shelf	(5.50, 1.00, 0.80)
13	shelf	(5.50, -1.00, 0.80)
14	charger	(0.30, 0.90, 0.07)

3.4 Code Implementation

The system is implemented as a Python Jupyter notebook, and it runs locally or in Colab. The excerpts below show the main parts of the notebook.

URDF generation (cell 3): each category is a short URDF string written to disk; the obstacle is shown.

```

OBSTACLE_URDF = """<?xml version="1.0"?>
<robot name="obstacle">
  <link name="base_link">
    <visual><geometry>
      <box size="0.5 0.5 0.5"/></geometry></visual>
    <collision><geometry>
      <box size="0.5 0.5 0.5"/></geometry></collision>
    <inertial><mass value="0"/></inertial>
  </link>
</robot>"""
URDF_PATHS["obstacle"] = write_urdf("obstacle.urdf", OBSTACLE_URDF)

```

Semantic layer and query (cell 5): semantic_db maps body ids to roles; get_nearest_target reads the robot's live pose and returns the closest target.

```

semantic_db = {}
for body_id, role, pos in object_ids:
    semantic_db[body_id] = {"role": role, "position": pos}

def get_nearest_target(robot_body_id):
    robot_pos, _ = p.getBasePositionAndOrientation(robot_body_id)
    targets = get_objects_by_role("target")
    best, best_dist = None, float("inf")
    for body_id, pos in targets:
        dist = math.dist(robot_pos, pos)
        if dist < best_dist:
            best_dist, best = dist, (body_id, pos)
    return {"body_id": best[0], "position": best[1],
            "distance": best_dist}

```

Camera rendering (cell 6): fixed eye and target pairs are rendered on the CPU in DIRECT mode; only the robot's pose changes between calls.

```

def render_view(camera_eye, camera_target, up_vector=(0, 0, 1)):
    view = p.computeViewMatrix(camera_eye, camera_target, up_vector)
    proj = p.computeProjectionMatrixFOV(
        fov=60, aspect=IMG_WIDTH / IMG_HEIGHT,
        nearVal=NEAR_PLANE, farVal=FAR_PLANE)
    _, _, rgb, _, _ = p.getCameraImage(
        IMG_WIDTH, IMG_HEIGHT, view, proj,
        renderer=p.ER_TINY_RENDERER)
    return np.reshape(rgb, (IMG_HEIGHT, IMG_WIDTH, 4))[:, :, :3]

```

3.5 Execution and Testing

I ran the notebook top to bottom in a local Python 3.12 environment with PyBullet 3.2.7. The completed notebook is hosted publicly on GitHub at github.com/Amaan9136/bsbi-assignments, under Sem2/Robotics Software and Programming/task1-code/Task1_PyBullet_Object_Semantics.ipynb, and opens in Google Colab via <https://colab.research.google.com/github/Amaan9136/bsbi->

[assignments/blob/main/Sem2/Robotics%20Software%20and%20Programming/task1-code/Task1_PyBullet_Object_Semantics.ipynb](#). All nine cells ran without errors.

Figure 2 shows the six rendered viewpoints and the robot pose in each. The cameras are fixed. Viewpoint 1 looks down on the whole room, and viewpoints 2 to 6 are oblique views from different sides. The robot stands at (0.0, 0.0), (3.0, 0.0), (5.5, 0.0), (1.5, 0.15), (3.0, 1.6) and (5.0, -1.5) in viewpoints 1 to 6. The overhead view shows every object and the robot. The other views show the same objects from the side.

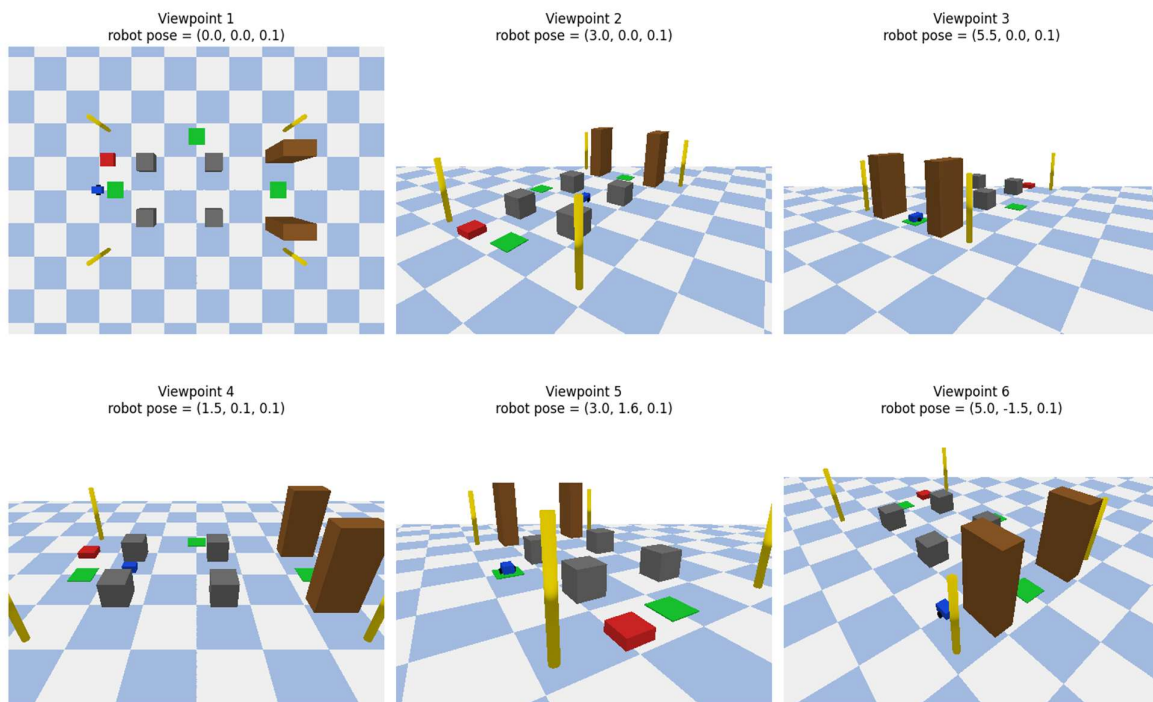


Figure 2: Six PyBullet camera viewpoints, robot repositioned between renders while all other objects remain fixed.

In cell 8 I moved the robot to (5.5, 0.0, 0.1), next to the drop-off target, and ran all three helper functions. `print_object_roles` printed the fourteen-row registry in Table 1. `get_objects_by_role("obstacle")` returned the four crate obstacles (ids 1 to 4) and `get_objects_by_role("shelf")` returned the two shelves (ids 12 and 13). `get_nearest_target` returned body id 7, the drop-off target at (5.50, 0.00, 0.01), at a distance of 0.092 m.

3.6 Evaluation and Reflection

The hardest part was keeping the camera framing consistent while the robot moved. My first version worked out each camera position from the robot's pose. The framing kept shifting, and in one configuration the robot left the frame completely. I changed the cameras to fixed

positions based on the room bounds and moved only the robot between renders. I then increased the views from three to six so the objects could be checked from more angles. I also tried assigning roles by matching URDF filenames to role strings when the objects were loaded. I rejected this because it was more error-prone. Passing the role explicitly in SCENE_LAYOUT keeps every role in one place where it is easy to check.

The main limitation is that I set the labels by hand. The robot cannot recognise a role from how an object looks. The brief asks for an interpretable environment and not a perception system, so this was acceptable for Task 1. An improvement would be to train a lightweight classifier on SceneNet RGB-D (McCormac et al., 2017) to predict a role from the images captured in Section 3.5. In a real warehouse, a wrong role could mean the robot drives into an obstacle or ignores a target, so I kept the link between geometry and role explicit.

Task 2: Goal-Oriented Autonomous Robot with Monitoring (ROS2)

4.1 Project Definition

The use case for Task 2 is a warehouse inspection patrol robot. A TurtleBot3 starts at the charging dock, visits a closed loop of four checkpoints around a warehouse bay, avoids pallet obstacles on the way, accepts simple operator commands and reports on its own performance. I chose this scenario because one mission covers goal navigation, obstacle avoidance, keyboard interaction and monitoring.

The package, `semantic_nav_monitor`, targets TheConstruct.ai running ROS2 Humble (Open-Source Robotics Foundation, 2024) with Gazebo Classic and the standard TurtleBot3 stack (ROBOTIS, 2023). I developed and tested it on TheConstruct.ai (TheConstruct, 2023) in Rosject `XctZpczWOA` (URL identifier 1051220, app.theconstruct.ai/desktop/rosject/1051220), workspace `ros2_ws`. The free tier does not give a permanent shareable URL, so I made the project public under the title `XctZpczWOA`, which is the unique name the brief asks for. I also used a local Ubuntu 24.04 workspace with ROS2 Jazzy and Gazebo Harmonic (Open Robotics, 2024) to view the LIDAR rays, the camera feed and the planned-path toggle outside the browser. The GitHub folder holds this Jazzy and Harmonic version. The Rosject copy uses a Gazebo Classic launch file and world. The local build was an extra check and not the submission platform.

The custom Gazebo world, `warehouse_inspection`, is a walled bay of 7 m by 7 m. It comes as a Classic `.world` for TheConstruct.ai and a Harmonic `.sdf` for local checks. It contains eight pallet stacks (two per patrol leg), three small pylons on the first leg, three coloured checkpoint markers and a green charging dock marker at the origin. The robot is a TurtleBot3 Burger with a forward camera (`burger_cam`). Its 360-degree LIDAR publishes `/scan` and its wheel odometry publishes `/odom`.

4.2 Design and Implementation

The package contains four ROS2 nodes (Quigley, Gerkey and Smart, 2015), summarised in Table 2. `mission_controller` is the only node that commands the robot. `monitor_node` only observes. `keyboard_hri_node` reads single keypresses without needing Enter. `path_visualizer_node` draws the planned route in the Gazebo Sim client and does not affect the

robot's behaviour. It needs the gz command line tool, so it works only in the Harmonic build.

Figure 3 shows the node and topic architecture.

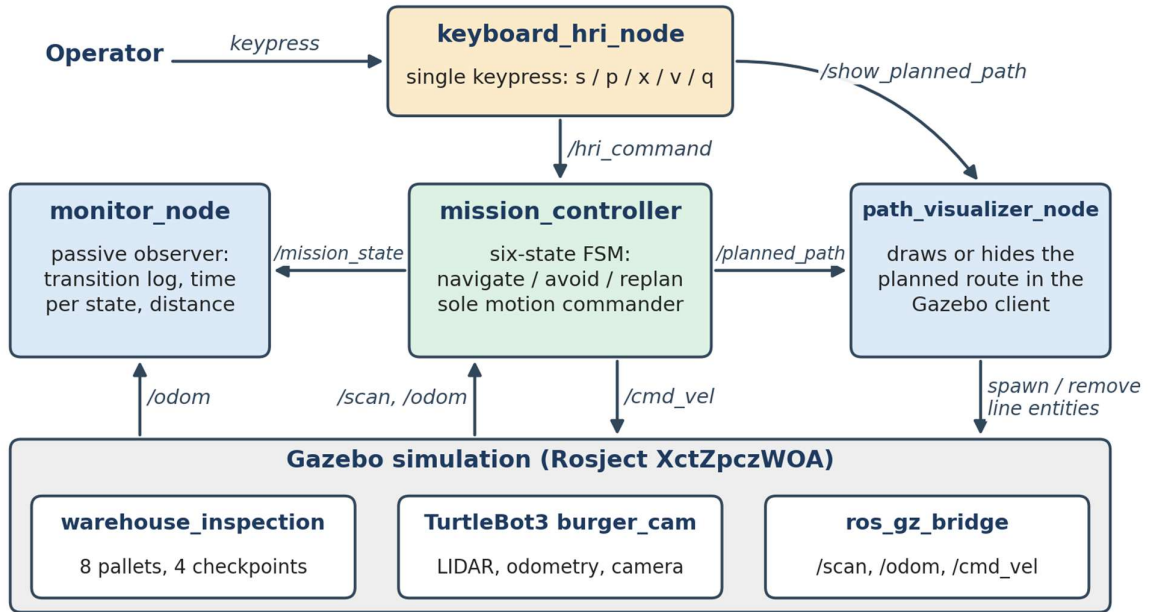


Figure 3: ROS2 node and topic architecture of the semantic_nav_monitor package, showing operator input, the four nodes and the Gazebo simulation.

Table 2: ROS2 node architecture.

Node	Subscribes	Publishes	Role
mission_controller	/scan, /odom, /hri_command	/cmd_vel, /mission_state, /planned_path	Six-state FSM: navigation, obstacle avoidance, replanning
monitor_node	/mission_state, /odom	(console log only)	Logs transitions; computes time-per-state, distance, obstacle count
keyboard_hri_node	terminal keypresses	/hri_command, /show_planned_path	Single-key HRI: s start, p pause, x stop, v toggle path, q quit
path_visualizer_node	/planned_path, /show_planned_path	spawns/removes Gazebo line entities	Draws or hides the planned route in the Gazebo client

mission_controller implements six states. The brief's minimum is five, and I added PLANNING ahead of NAVIGATE. Table 3 lists each state's trigger and behaviour as defined in the MissionState enum. Figure 4 shows the transitions.

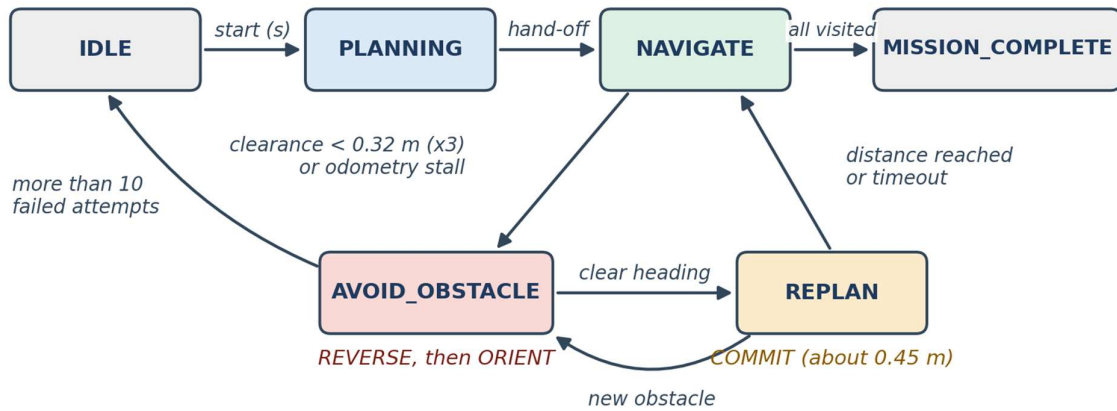


Figure 4: Six-state finite state machine in mission_controller, with the REVERSE, ORIENT and COMMIT sub-phases shown beneath their parent states.

Table 3: Finite state machine (six states).

State	Entered when	Behaviour
IDLE	At launch, or after the mission aborts	Stationary, waiting for a start command
PLANNING	start received	Builds the four-checkpoint route as a nav_msgs/Path, publishes it once, then hands off immediately
NAVIGATE	After planning, or once an obstacle is cleared	Proportional controller drives towards the next checkpoint
AVOID_OBSTACLE	Forward clearance below 0.32 m for three consecutive control cycles, or an odometry-based stall	Reverses if closer than 0.20 m (REVERSE), then rotates to a LIDAR-probed clear heading (ORIENT)
REPLAN	A clear heading has been reached	Drives forward roughly 0.45 m along that heading (COMMIT) before resuming Navigate
MISSION_COMPLETE	All four checkpoints visited, or stop received	Robot halts; final state published

I made two design decisions that need explaining. First, obstacle avoidance has three phases (REVERSE, ORIENT and COMMIT). The robot probes escape headings at eleven angles per

side using LIDAR and scores each by goal alignment. Among headings with similar alignment it takes the one with more open space. It accepts a heading only with at least 0.21 m of clear runway, which is the robot's half-width (0.11 m) plus a 0.05 m lateral margin and a further 0.05 m. An odometry stall check runs alongside. If the robot moves less than 0.05 m in one second of commanded forward motion, it counts as blocked whatever the LIDAR says. After ten failed avoidance attempts the mission aborts to IDLE and needs a new start command. Second, in the Jazzy and Harmonic build the velocity commands are TwistStamped, because the `ros_gz_bridge` started by the TurtleBot3 launch files expects that type. An extra bridge in an earlier launch file subscribed to `/cmd_vel` as plain Twist. The type mismatch stopped the robot moving until that bridge was removed.

`monitor_node` subscribes to `/mission_state` and `/odom`. On every state change it logs the transition with a timestamp and adds the elapsed time to the previous state. Each time the robot enters AVOID_OBSTACLE it increments an obstacle-encounter counter. The code comments note that this counter stands in for the brief's collisions metric, since the robot avoids contact. Odometry feeds a running distance total. A five-second timer prints a summary during the mission, and a final summary is printed on MISSION_COMPLETE. Times use the system clock, so they are wall-clock seconds. `path_visualizer_node` listens on `/planned_path`. When told to show the path, it spawns thin static box entities along the route through the Gazebo EntityFactory service. The 'v' key or a native GUI panel toggles the line without changing the plan.

4.3 Execution and Testing

I built the package in the Rosject with `colcon build --packages-select semantic_nav_monitor` and sourced the workspace. After setting `TURTLEBOT3_MODEL=burger_cam` I ran `ros2 launch semantic_nav_monitor semantic_nav_monitor.launch.py` in Shell 1 and `ros2 run semantic_nav_monitor keyboard_hri_node` in Shell 2. Figure 5 shows the Rosject workspace. The file explorer shows the Rosject name XctZpczWOA, and `mission_controller.py` is open in the code editor. The Simulation panel shows the warehouse world with the robot, eight pallets and four checkpoints. A shell is printing `monitor_node`'s summary.

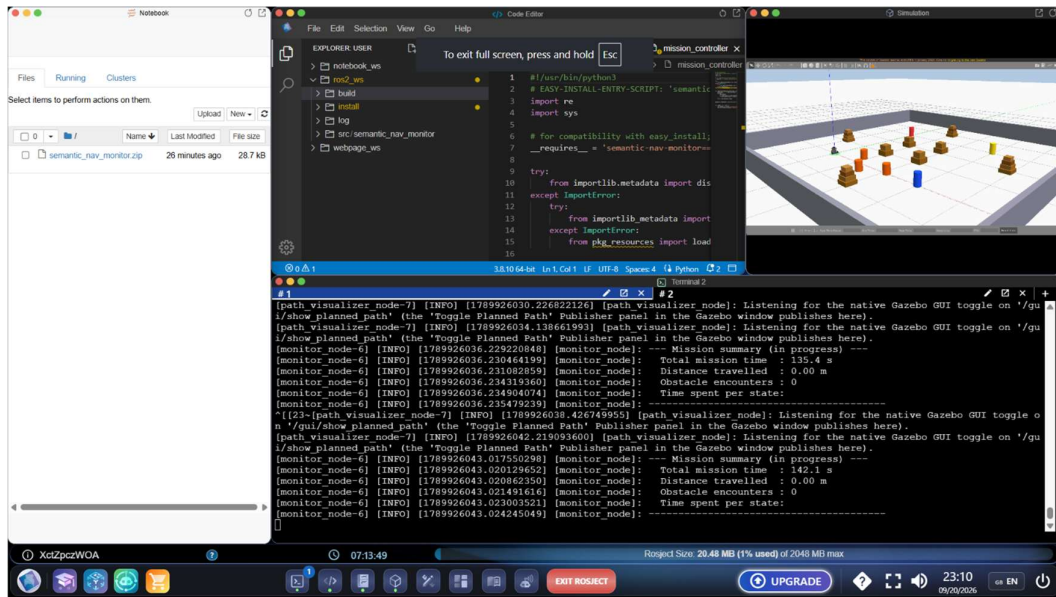


Figure 5: TheConstruct.ai Rosject XctZpczWOA: file explorer, mission_controller.py in the code editor, the live Simulation panel, and monitor_node's summary stream in the shell.

Figure 6 shows the Gazebo client during the run. The status bar shows 00:02:11 of real time, 94,858 iterations and a real-time factor of 0.70, so the simulation was running.

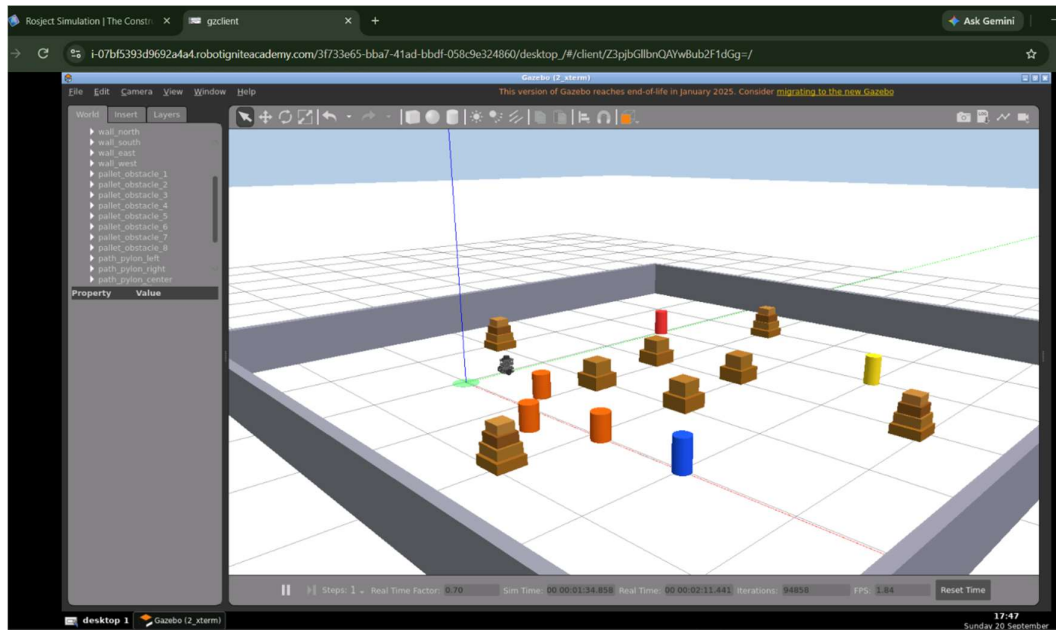


Figure 6: Gazebo client on TheConstruct.ai: robot near the charging dock among the eight pallet obstacles and checkpoint markers, with elapsed real time and iteration count visible.

Figure 7 shows keyboard_hri_node in a Rosject shell next to the Simulation panel. The s and p keypresses are echoed as Published HRI command: 'start' and 'pause', and Planned-path line: ON confirms that the path toggle also responded.

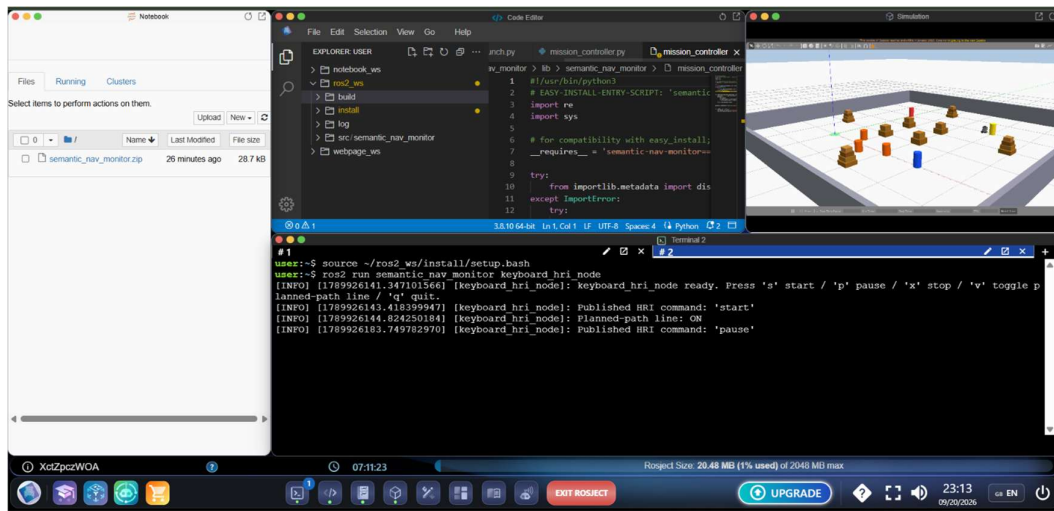


Figure 7: keyboard_hri_node in a Rosject shell: start and pause keypresses published and confirmed, alongside the live Simulation panel.

Figure 8 is a live ros2 topic echo /mission_state capture. Reading oldest to newest, it records AVOID_OBSTACLE, REPLAN, AVOID_OBSTACLE, IDLE, PLANNING, NAVIGATE and finally MISSION_COMPLETE. The code only returns to IDLE from AVOID_OBSTACLE after ten failed avoidance attempts, so the mission most likely aborted and I restarted it with s. It then planned again and reached MISSION_COMPLETE.

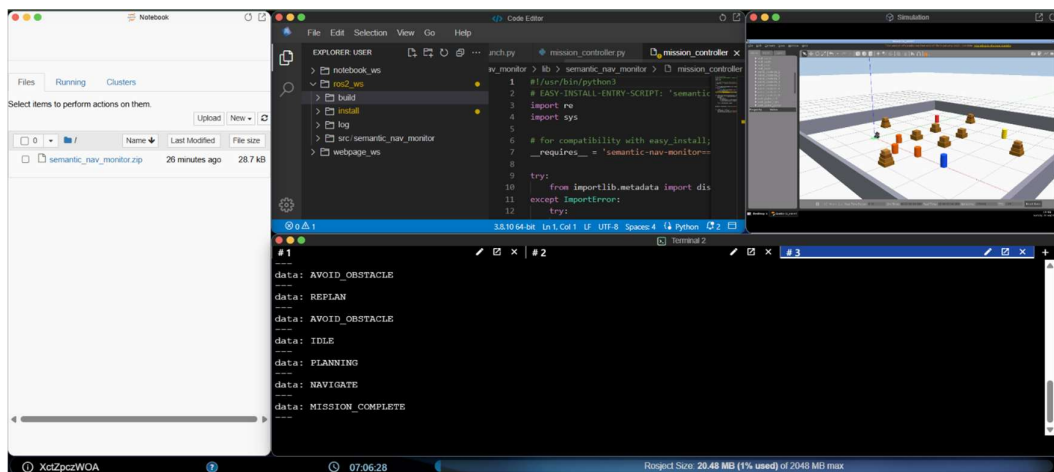


Figure 8: Live /mission_state echo showing the robot cycle through AVOID_OBSTACLE, REPLAN, IDLE, PLANNING, NAVIGATE and MISSION_COMPLETE.

Figure 9 is monitor_node's summary from partway through the run, and Table 4 copies its values. The header still says "in progress", so this is not the final summary. The state times add up to 133.5 s against a total of 172.8 s. monitor_node only adds a state's time when the robot leaves that state, so the state running at the moment of capture is missing.

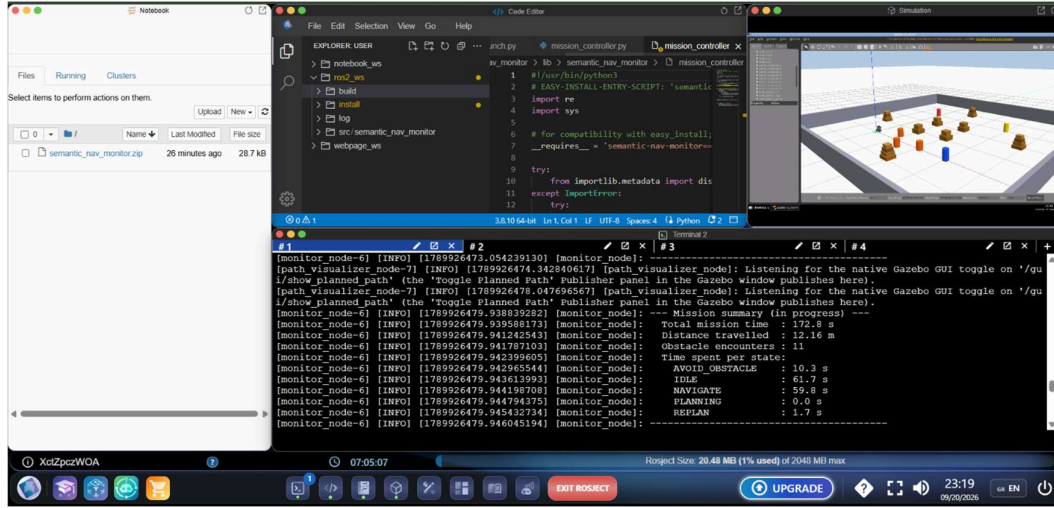


Figure 9: monitor_node's mission summary, captured while the mission was in progress.

Table 4: Mission summary logged by monitor_node on TheConstruct.ai, Rosject XctZpczWOA, while the mission was in progress.

Metric	Value
Total mission time	172.8 s
Distance travelled	12.16 m
Obstacle encounters	11
Time in AVOID_OBSTACLE	10.3 s
Time in IDLE	61.7 s
Time in NAVIGATE	59.8 s
Time in PLANNING	0.0 s
Time in REPLAN	1.7 s

The local Jazzy/Harmonic workspace gave one extra view that is hard to capture in the Rosject browser window. Figure 10 shows the LIDAR rays near a pallet obstacle with the forward camera feed beside them. This is the sensing data the AVOID_OBSTACLE logic in Section 4.2 works from.

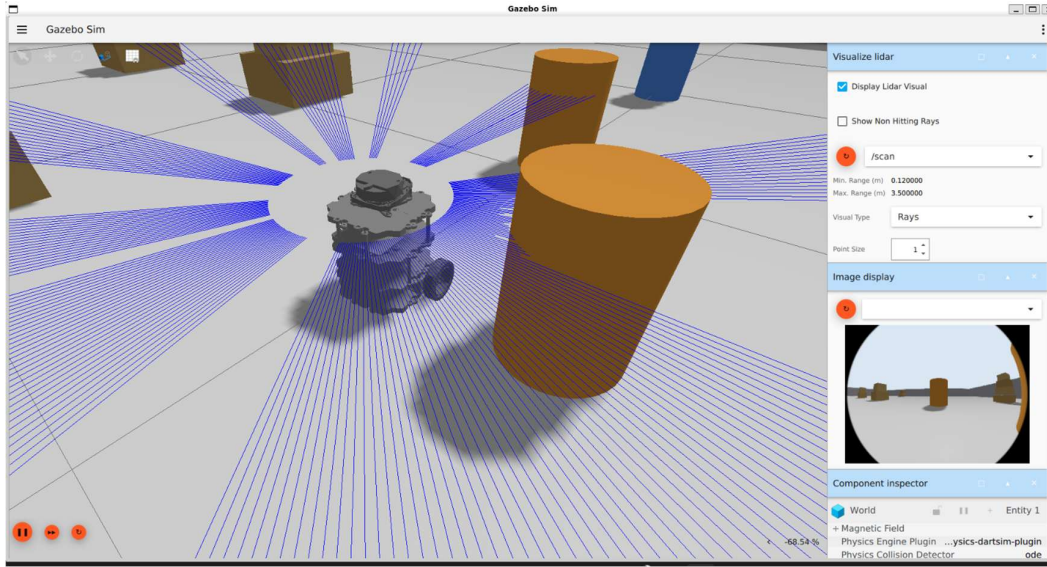


Figure 10: Local Gazebo Harmonic verification: LIDAR rays visualised around the robot near a pallet obstacle, with the forward camera feed shown alongside.

The Rosject evidence has five parts. These are the Rosject identity and live simulation (Figure 5), the running world (Figure 6), keyboard HRI with acknowledgement (Figure 7), a state sequence ending in MISSION_COMPLETE (Figure 8) and the logged metrics (Figure 9 and Table 4). The LIDAR view in Figure 10 comes from the local workspace.

4.4 Evaluation and Reflection

The main problem in development was over-cautious obstacle avoidance. The robot took wide detours around gaps it could have fitted through. The code comments show the cause. The margin was 0.09 m on top of the half-width, so the robot treated itself as about 0.20 m wide. I cut it to 0.05 m. My first scoring rule also picked the heading closest to the goal even with little clearance, so the robot oscillated at gap edges. I changed it to prefer the heading with more open space among similar ones, and I retested after each change.

Table 4 shows eleven obstacle encounters over 12.16 m, with 10.3 s in AVOID_OBSTACLE against 59.8 s in NAVIGATE. The 61.7 s in IDLE most likely includes the wait before the first start and the abort in Figure 8.

REPLAN is also simple. It drives forward along a clear heading for 0.45 m and does not plan around the whole obstacle. A local costmap-based planner, as used within the Nav2 navigation stack (Macenski et al., 2020), would route around larger obstacles better, but it is much more work. The pause key also has no resume, because s is only accepted from IDLE.

TheConstruct.ai's free tier gives no permanent shareable URL, so the Rosject is shared by its name, XctZpczWOA.

Explicit states, reactive avoidance with an odometry backup and constant logging are easy to explain to warehouse staff (Siegwart, Nourbakhsh and Scaramuzza, 2011). The figures in Table 4 are the kind an operator could check.

Combined Evaluation and Reflection

Task 1 gives every object in a static scene a role. I assigned the roles by hand, and the code retrieves them with dictionary lookups, as Table 1 and the nearest-target result in Section 3.5 show. In Task 2 the robot moves between six behaviour states, such as avoiding an obstacle, replanning and completing the mission. It reports which state it is in as it goes.

In both tasks a data step fed a decision step. In Task 1 the queries read object positions from PyBullet and semantic_db. In Task 2 the LIDAR and odometry readings drove the state machine. Both were easier to check because the system reported what it was doing. The most useful lesson came from Task 2. The avoidance was too cautious and took several rounds of threshold tuning to fix. A robot can avoid every obstacle and still behave badly, and the monitoring output helped me see that.

Concluding Remarks

Task 1 produced a PyBullet room with fourteen labelled objects and six camera views. The role lookups worked as intended, including the nearest-target query, which returned 0.092 m. Task 2 produced a TurtleBot3 patrol in ROS2 that reached MISSION_COMPLETE in Rosject XctZpczWOA. That run was not clean. The state log shows the robot dropping back to IDLE once, and I restarted it. Table 4 also comes from a single run, so I cannot say how repeatable the figures are. With more time I would record an uninterrupted run with the final monitor summary. I would also add a resume key, since pause currently has none. For Task 1 I would try predicting roles from the rendered images. Both tasks taught me more from what went wrong than from what worked first time. The camera framing and the over-cautious avoidance each took several rounds of changes before the results were usable.

Acknowledgement

Perplexity AI (2026) was used to support literature searching, identify potentially relevant academic sources. *All sources included in the reference list were checked by the author and cited as the original academic works rather than citing Perplexity AI as a source. The analysis, interpretation, discussion, and final written content submitted in this report are the author's own work and responsibility.*

The architecture and pipeline diagrams in Sections 3.3 and 4.2 were generated using Python scripts with the matplotlib library, written specifically to render the structures described in this report. The scripts and their outputs were reviewed by the author before inclusion.

References

Collins, J., Chand, S., Vanderkop, A. and Howard, D. (2021) ‘A Review of Physics Simulators for Robotic Applications.’ IEEE Access 9: 51416-51431. [online] Available from <https://doi.org/10.1109/ACCESS.2021.3068769> [Accessed 20 September 2026]

Coumans, E. and Bai, Y. (2021) PyBullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning. [online] Available from <https://github.com/bulletphysics/bullet3> [Accessed 20 September 2026]

Duckett, T., Pearson, S., Blackmore, S. and Grieve, B. (2018) Agricultural Robotics: The Future of Robotic Agriculture. UK-RAS White Papers. [online] Available from <https://arxiv.org/abs/1806.06762> [Accessed 20 September 2026]

Kostavelis, I. and Gasteratos, A. (2015) ‘Semantic mapping for mobile robotics tasks: A survey.’ Robotics and Autonomous Systems 66: 86-103. [online] Available from <https://doi.org/10.1016/j.robot.2014.12.006> [Accessed 20 September 2026]

Macenski, S., Martin, F., White, R. and Gines Clavero, J. (2020) ‘The Marathon 2: A Navigation System.’ In: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Las Vegas, 24-30 October 2020: 2718-2725

McCormac, J., Handa, A., Leutenegger, S. and Davison, A.J. (2017) ‘SceneNet RGB-D: Can 5M Synthetic Images Beat Generic ImageNet Pre-training on Indoor Segmentation?’ In: Proceedings of the IEEE International Conference on Computer Vision (ICCV). Venice, 22-29 October 2017: 2678-2687

Open-Source Robotics Foundation (2024) ROS 2 Documentation: Humble. [online] Available from <https://docs.ros.org/en/humble/index.html> [Accessed 20 September 2026]

Open Robotics (2024) Gazebo Harmonic Documentation. [online] Available from <https://gazebo.org/docs/harmonic> [Accessed 20 September 2026]

Perplexity AI (2026) Perplexity [AI tool]. [online] Available from <https://www.perplexity.ai> [Accessed 20 September 2026]

PyBullet (2023) PyBullet Quickstart Guide. [online] Available from <https://pybullet.org/wordpress/> [Accessed 20 September 2026]

Quigley, M., Gerkey, B. and Smart, W.D. (2015) Programming Robots with ROS: A Practical Introduction to the Robot Operating System. Sebastopol, CA: O'Reilly Media

ROBOTIS (2023) TurtleBot3 e-Manual. [online] Available from <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> [Accessed 20 September 2026]

Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) Introduction to Autonomous Mobile Robots. 2nd ed. Cambridge, MA: MIT Press

TheConstruct (2023) ROS2 Basics in 5 Days (Python). [online] Available from <https://www.theconstruct.ai/> [Accessed 20 September 2026]